

Module 6

Complex Testbenches

Learning objectives

- Build production-quality multi-agent testbenches

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["Testbench Architecture"]

T2["Multi-Agent Environments"]

T1 --> T2

T3["Protocol Verification"]

T2 --> T3

Build production-quality multi-agent testbenches

Overview

- This module covers building complex, production-quality testbenches with multiple agents, protocol...

Syllabus topics

1. Testbench Architecture (1/13)

- Hierarchical Component Organization
- Component hierarchy design
- Separation of concerns
- Component relationships
- Environment structure
- Component Integration

1. Testbench Architecture (2/13)

- Agent construction
- Environment assembly
- Component connections
- Phase-based execution
- Best Practices
- Clean component hierarchy

1. Testbench Architecture (3/13)

- Reusable components
- Configuration management
- Maintainable testbench structure
- Transaction Class (``ArchTxn``): Base transaction with address, data, and write/read control
- Sequencer (``ArchSequencer``): UVM sequencer for transaction flow control
- Driver (``ArchDriver``): UVM driver that receives and drives transactions

1. Testbench Architecture (4/13)

- Monitor (``ArchMonitor``): UVM monitor that observes DUT behavior
- Agent (``ArchAgent``): UVM agent containing sequencer, driver, and monitor
- Environment (``ArchEnv``): UVM environment containing agent instances
- Sequence (``ArchSmokeSeq``): UVM sequence generating random transactions
- Test (``ArchitectureTest``): Top-level UVM test orchestrating verification
- Method Signature: ``function bit randomize();``

1. Testbench Architecture (5/13)

- Return Type: ``bit`` (1 = success, 0 = failure)
- Purpose: Generate random test vectors with constraints
- Usage: Call after creating transaction, before sending to sequencer
- Key: ``void'()` suppresses Verilator warning about ignored return value
- Constraints: Use ``with { constraint_expression; }`` for conditional randomization
- Example: ``void'(t.randomize() with { addr[31:16] == 16'h0000; });`` constrains upper address bits

1. Testbench Architecture (6/13)

- Method Signature: ``task body();`` (no parameters, no return)
- Purpose: Generate and send multiple transactions to sequencer
- Execution Flow:
- Key: ``repeat`` loop generates multiple transactions sequentially
- Blocking Behavior: ``start_item()`` and ``finish_item()`` both block
- Transaction Lifetime: Transaction created in loop, sent to sequencer, then driver processes it

1. Testbench Architecture (7/13)

- Method Signature: ``function void build_phase(uvm_phase phase);``
- Phase Type: Build-time phase (function, executes top-down)
- Purpose: Create all child components of the agent
- Execution Order:
- Key: All components created in agent's ``build_phase()`, before `connect_phase()`,`
- Factory Benefits: Enables type/instance overrides via factory

1. Testbench Architecture (8/13)

- Hierarchical Naming: Full path =
"test.env.agent.seqr" (automatic)
- Method Signature: ``function void
connect_phase(uvm_phase phase);``
- Phase Type: Connect-time phase (function, executes
bottom-up)
- Purpose: Connect TLM ports/exports between
components
- Execution Order:
- Key: TLM connections made in ``connect_phase()``,
after all components created

1. Testbench Architecture (9/13)

- TLM Pattern: Port (initiator) connects to Export (target)
- Transaction Flow: Sequence → Sequencer (export) → Driver (port) → DUT
- Connection Timing: Must happen after ``build_phase()` (components must exist)
- Method Signature: ``task run_phase(uvm_phase phase);``
- Phase Type: Run-time phase (task, executes concurrently)
- Purpose: Execute test stimulus (sequences, driving, monitoring)

1. Testbench Architecture (10/13)

- Execution Flow:
- Key: Objection mechanism controls phase completion
- Sequence Execution: ``seq.start(sequencer)`` blocks until ``body()`` completes
- Transaction Flow: Test → Sequence → Sequencer → Driver → DUT
- Timing: All `run_phase` tasks execute concurrently (forked)
- Hierarchical component organization enables maintainable testbenches

1. Testbench Architecture (11/13)

- Separation of concerns (driver, monitor, sequencer)
improves reusability
- Component factory and configuration provide flexibility
- Phase-based execution model ensures proper initialization and execution
- Transaction-based communication decouples components
- Component hierarchy: Test → Env → Agent → Sequencer/Driver/Monitor
- Factory creation: ``type_id::create()`` for all components

1. Testbench Architecture (12/13)

- Phase execution: Build → Connect → Run → Report
- Sequence execution: ``seq.start(sequencer)`` in `run_phase`
- UVM component hierarchy construction
- Transaction generation and driving
- Component phase execution
- UVM report summary with component counts

1. Testbench Architecture (13/13)

- Component Hierarchy: Agent → Sequencer/Driver/Monitor structure
- Factory Pattern: Using ``type_id::create()`` for component creation
- Phase Execution: Build, connect, run, check, report phases
- Transaction Flow: Sequence → Sequencer → Driver → DUT

2. Multi-Agent Environments (1/12)

- Multiple Agent Coordination
- Multiple agent instantiation
- Agent configuration
- Agent communication
- Cross-agent stimulus
- Virtual Sequences

2. Multi-Agent Environments (2/12)

- Virtual sequence definition
- Multi-sequencer coordination
- Parallel sequence execution
- Sequence synchronization
- Agent Reuse
- Parameterized agents

2. Multi-Agent Environments (3/12)

- Agent configuration database
- Reusable agent patterns
- Agent instantiation patterns
- Transaction (``MA_Txn``): Transaction with agent ID and data fields
- Agent (``MA_Agent``): Parameterized agent with configurable agent ID
- Virtual Sequence (``MA_VirtualSeq``): Coordinates sequences across multiple sequencers

2. Multi-Agent Environments (4/12)

- Environment (``MA_Env``): Contains two agent instances (a0 and a1)
- Test (``MultiAgentTest``): Orchestrates multi-agent verification
- Method Signature: ``static function bit get(uvm_component cntxt, string inst_name, string field_name...`
- Template Parameter: ``T`` = type of value (int, string, object, etc.)
- Purpose: Retrieve configuration values from ConfigDB
- Search Behavior: Searches up component hierarchy from ``cntxt``

2. Multi-Agent Environments (5/12)

- Usage: Call in ``build_phase()` before using the value
- Key: Always check return value, provide defaults
- Hierarchy Search: Searches current component → parent → ... → top
- Method Signature: ``static function void set(uvm_component cntxt, string inst_name, string field_name...`
- Template Parameter: ``T`` = type of value
- Purpose: Store configuration values in ConfigDB

2. Multi-Agent Environments (6/12)

- Timing: MUST be called BEFORE creating the component
- Hierarchical Path: Full path = "env.a0.agent_id"
(automatic)
- Key: Set config → Create component → Component
retrieves config in its ``build_phase()``
- Pattern: Environment sets config, agents retrieve
config
- Method Signature: ``task
execute_item(uvm_sequence_item item);``
- Purpose: Execute transaction directly on sequencer
(combines `start_item` + `finish_item`)

2. Multi-Agent Environments (7/12)

- Behavior:
- Sends transaction to sequencer
- Waits for driver to complete
- Blocks until transaction fully processed
- Usage: Alternative to ``start_item()` + ``finish_item()` pair
- Key: Useful in virtual sequences for direct execution on multiple sequencers

2. Multi-Agent Environments (8/12)

- Parallel Execution: Fork-join enables concurrent execution on multiple sequencers
- Transaction Flow: Virtual sequence → Multiple sequencers → Multiple drivers → DUT
- Method Signature: ``task start(uvm_sequencer_base sequencer);``
- Purpose: Start virtual sequence execution
- Virtual Sequence Pattern:
- Key: Sequencer references MUST be set before calling ``start()``

2. Multi-Agent Environments (9/12)

- Difference: Regular sequences use ``start(sequencer)``, virtual sequences use assigned references
- Execution: Virtual sequence `body()` uses assigned sequencers for parallel execution
- Purpose: Execute transactions on multiple agents concurrently
- Key: Fork-join enables parallel execution
- Multiple agents enable complex verification scenarios
- Virtual sequences coordinate stimulus across agents

2. Multi-Agent Environments (10/12)

- Agent configuration via config database provides flexibility
- Parallel sequence execution using fork-join enables concurrent testing
- Agent reuse patterns improve productivity
- Agent configuration - Via ConfigDB before creation
- Virtual sequence - Coordinates multiple sequencers
- ``execute_item()`` - Direct transaction execution

2. Multi-Agent Environments (11/12)

- Parallel execution - Fork-join for concurrent agents
- Agent reuse - Same agent class, different instances
- Two agents running in parallel
- Virtual sequence coordinating both agents
- Agent-specific transaction generation
- Cross-agent stimulus coordination

2. Multi-Agent Environments (12/12)

- Virtual Sequences: Sequences that coordinate multiple sequencers
- Agent Configuration: Using ``uvm_config_db`` to configure agent parameters
- Parallel Execution: Fork-join for concurrent sequence execution
- Agent Reuse: Same agent class used for multiple instances

3. Protocol Verification (1/9)

- Protocol Modeling
- Interface definition
- Protocol transaction modeling
- Protocol signal mapping
- Protocol timing
- Protocol Sequences

3. Protocol Verification (2/9)

- Protocol-aware sequences
- Protocol transaction generation
- Protocol sequence patterns
- Protocol coverage
- Protocol Drivers and Monitors
- Protocol driver implementation

3. Protocol Verification (3/9)

- Protocol monitor implementation
- Handshake protocol handling
- Protocol signal sampling
- Interface (``axi_lite_if``): AXI4-Lite interface with all required signals
- Transaction (``AxiTxn``): AXI transaction with read/write, address, and data
- Driver (``AxiDriver``): Protocol-aware driver implementing AXI4-Lite handshake protocol

3. Protocol Verification (4/9)

- Monitor (``AxiMonitor``): Protocol monitor for observing AXI transactions
- Environment (``ProtocolEnv``): Protocol verification environment
- Test (``ProtocolTest``): Protocol verification test with AXI4-Lite slave DUT
- Purpose: Get virtual interface for protocol access
- Key: Must be set before driver can access signals
- Purpose: Implement AXI write protocol

3. Protocol Verification (5/9)

- Key: Handshake pattern - assert VALID, wait for READY
- Purpose: Implement AXI read protocol
- Key: Capture read data from interface
- Purpose: Generic handshake implementation
- Key: Assert VALID, wait for READY, deassert VALID
- Purpose: Synchronize with protocol clock

3. Protocol Verification (6/9)

- Key: All protocol operations clock-synchronized
- Purpose: Set interface for all components
- Key: Set before ``run_test()`
- Protocol-shaped driver/monitor scaffolding enables protocol verification
- Handshake protocol implementation (VALID/READY) is fundamental
- Interface-based communication provides structured protocol access

3. Protocol Verification (7/9)

- Protocol transaction modeling captures protocol semantics
- Clock-synchronized protocol operations ensure correct timing
- Virtual interface - Access protocol signals
- Handshake pattern - VALID/READY protocol
- Clock synchronization - `@(posedge clk)` for all operations
- Write protocol - Address + Data + Response channels

3. Protocol Verification (8/9)

- Read protocol - Address + Data channels
- `do-while` loop - Wait for handshake completion
- AXI4-Lite write and read transactions
- Handshake protocol demonstration
- Protocol driver and monitor operation
- DUT interaction with protocol interface

3. Protocol Verification (9/9)

- Protocol Interfaces: Structured interface definition for protocol signals
- Handshake Protocol: VALID/READY handshake for flow control
- Protocol Drivers: Clock-synchronized protocol signal driving
- Protocol Monitors: Protocol signal observation and transaction reconstruction

4. Protocol Checkers (1/8)

- Protocol Rule Checking
- Protocol rule definition
- Assertion-like checking
- Bounded wait checking
- Protocol compliance verification
- Error Detection

4. Protocol Checkers (2/8)

- Protocol violation detection
- Error reporting
- Protocol timing checks
- Protocol state checking
- Checker Implementation
- UVM checker components

4. Protocol Checkers (3/8)

- Interface monitoring
- Rule enforcement
- Compliance reporting
- Interface (``chk_if``): Simple handshake interface with valid/ready signals
- Checker Component (``CheckerComp``): UVM component that monitors protocol rules
- Protocol Rules: Valid must stay asserted until ready (bounded wait)

4. Protocol Checkers (4/8)

- Test (``ProtocolCheckersTest``): Test that exercises protocol checker
- Purpose: Monitor protocol compliance
- Key: Bounded wait prevents infinite waits
- Purpose: Check that protocol doesn't wait indefinitely
- Key: Count cycles, report if exceeded
- Purpose: Ignore protocol during reset

4. Protocol Checkers (5/8)

- Key: Use ``continue`` to skip reset cycles
- Purpose: Detect protocol states
- Key: Check signal combinations
- Purpose: Report protocol violations
- Key: Use appropriate severity level
- Protocol rule checking ensures protocol compliance

4. Protocol Checkers (6/8)

- Assertion-like patterns in UVM provide structured checking
- Bounded wait checking prevents protocol violations
- Protocol compliance verification catches design errors
- Error detection and reporting enable debugging
- Bounded wait - Check timeout conditions
- Reset handling - Skip checking during reset

4. Protocol Checkers (7/8)

- State monitoring - Check protocol signal combinations
- Error reporting - Report violations with ``uvm_error``
- Forever loop - Continuous monitoring
- Protocol handshake monitoring
- Rule violation detection (if any)
- Bounded wait checking

4. Protocol Checkers (8/8)

- Protocol compliance reporting
- Protocol Checkers: UVM components that monitor protocol rules
- Bounded Wait: Checking that protocol signals don't wait indefinitely
- Rule Enforcement: Detecting and reporting protocol violations
- Compliance Verification: Ensuring design follows protocol specification

5. Multi-Channel Scoreboards (1/17)

- Multi-Channel Scoreboarding
- Multiple stream handling
- Transaction matching
- Result aggregation
- Scoreboard architecture
- Transaction Matching

5. Multi-Channel Scoreboards (2/17)

- Matching algorithms
- Transaction comparison
- Match criteria
- Unmatched transaction handling
- Result Aggregation
- Pass/fail counting

5. Multi-Channel Scoreboards (3/17)

- Result reporting
- Statistics collection
- Coverage tracking
- Transaction (``SB_Txn``): Scoreboard transaction with stream ID, sequence number, and data
- Stream Producer (``StreamProducer``): Component that generates transactions for a stream
- Multi-Channel Scoreboard (``MultiChannelScoreboard``): Scoreboard that matches transactions from mult...

5. Multi-Channel Scoreboards (4/17)

- Test (`ScoreboardsTest`): Test with two stream producers and a scoreboard
- Purpose: Match transactions from multiple streams
- Key: Use associative arrays indexed by sequence number
- Purpose: Manage transaction queues per stream
- Key: Associative arrays enable efficient lookup
- Purpose: Match transactions by sequence number

5. Multi-Channel Scoreboards (5/17)

- Key: Compare data, track pass/fail, clean up
- Purpose: Generate transactions for a stream
- Key: Assign sequence numbers for matching
- Purpose: Connect multiple streams to scoreboard
- Key: One scoreboard can receive from multiple sources
- Purpose: Report final results

5. Multi-Channel Scoreboards (6/17)

- Key: Aggregate pass/fail counts
- Multi-channel scoreboarding enables complex verification scenarios
- Transaction matching across streams verifies data integrity
- Result aggregation provides overall test status
- Scoreboard architecture supports multiple verification paths
- Stream-based verification enables parallel checking

5. Multi-Channel Scoreboards (7/17)

- Associative arrays - Store transactions by sequence number
- ``exists(key)`` - Check if both streams have transaction
- ``delete(key)`` - Remove matched transactions
- Transaction matching - Match by sequence number, compare data
- Multiple streams - One scoreboard, multiple producers
- Result aggregation - Track pass/fail across all matches

5. Multi-Channel Scoreboards (8/17)

- Two stream producers generating transactions
- Scoreboard matching transactions from both streams
- Pass/fail reporting
- Transaction matching results
- Multi-Channel Scoreboards: Scoreboards that handle multiple transaction streams
- Transaction Matching: Matching transactions based on sequence numbers or other criteria

5. Multi-Channel Scoreboards (9/17)

- Result Aggregation: Collecting and reporting overall test results
- Stream-Based Verification: Verifying data integrity across multiple paths
- Purpose: Create components using UVM factory
- Usage: In ``build_phase()` of parent component
- Key: Always provide name and parent
- Purpose: Connect TLM ports/exports

5. Multi-Channel Scoreboards (10/17)

- Usage: In ``connect_phase()` after build
- Key: Connect driver to sequencer
- Purpose: Execute sequence on sequencer
- Usage: In test's ``run_phase()`
- Key: Create sequence, then start on sequencer
- Purpose: Configure agent parameters

5. Multi-Channel Scoreboards (11/17)

- Usage: Before creating agent
- Key: Set config, then create component
- Purpose: Retrieve agent configuration
- Usage: In agent's ``build_phase()`
- Key: Check return value, provide defaults
- Purpose: Execute transaction directly on sequencer

5. Multi-Channel Scoreboards (12/17)

- Usage: Alternative to ``start_item()` / ``finish_item()`
- Key: Useful in virtual sequences for direct execution
- Note: Transaction must be created and randomized first
- Purpose: Coordinate multiple sequencers
- Key: Set sequencer references before starting
- Purpose: Get virtual interface for protocol access

5. Multi-Channel Scoreboards (13/17)

- Usage: In driver/monitor ``build_phase()`
- Key: Must be set before component can access signals
- Purpose: Set interface for all components
- Usage: In module before ``run_test()`
- Key: Use wildcard ``"*"` to match all components
- Purpose: Implement VALID/READY handshake

5. Multi-Channel Scoreboards (14/17)

- Key: Assert VALID, wait for READY, deassert VALID
- Purpose: AXI write transaction
- Key: Drive address and data, wait for both ready
- Purpose: AXI read transaction
- Key: Drive address, wait for ready, capture data
- Purpose: Check protocol doesn't wait indefinitely

5. Multi-Channel Scoreboards (15/17)

- Key: Count cycles, report if exceeded bound
- Purpose: Ignore protocol during reset
- Key: Use `continue` to skip reset cycles
- Purpose: Continuously monitor protocol
- Key: Forever loop with clock synchronization
- Purpose: Store transactions by sequence number

5. Multi-Channel Scoreboards (16/17)

- Key: Use sequence number as index
- Purpose: Check if transaction exists
- Key: Use ``exists()`` method
- Purpose: Remove matched transactions
- Key: Use ``delete()`` method
- Purpose: Match transactions across streams

5. Multi-Channel Scoreboards (17/17)

- Key: Check both streams, compare, clean up
- Purpose: Connect multiple streams to one scoreboard
- Key: One scoreboard can receive from multiple sources

Hands-on examples

Module 6 self-check

```
$ ./scripts/module6.sh --check
```

Module 6 self-check
(run ./scripts/module6.sh when ready)

Exercise scaffold

```
./scripts/module6.sh --scaffold
```

Practice & assessment

Exercises

- Testbench Architecture
- Multi-Agent Coordination
- Protocol Verification
- Protocol Checkers
- Multi-Channel Scoreboards

Assessment checklist

- Can design complex testbench architectures
- Can coordinate multiple agents
- Can verify protocols
- Can implement protocol checkers
- Can build multi-channel scoreboards
- Can apply production-quality testbench patterns

Summary & next steps

- Build production-quality multi-agent testbenches
- Complete module6/CHECKLIST.md
- Run `./scripts/module6.sh --check`
- Next: Next module in course